

SmartLink AI

Complete Backend Guide — v2.0 (with Auth + QR Code)

What's New in v2.0

This version adds 2 features on top of the original backend:

- User Authentication — register, login, JWT-protected routes, each user owns their links
- QR Code Generation — every short link gets a downloadable QR code image

Everything from the original backend (Redis, Groq AI, rate limiting, malicious URL check) is preserved and unchanged.

1. COMPLETE FOLDER STRUCTURE

```
smartlink-ai/
├── server/
│   ├── index.js
│   ├── config/
│   │   ├── db.js
│   │   └── redis.js
│   ├── models/
│   │   ├── Url.js
│   │   ├── User.js ← NEW
│   │   └── routes/
│   │       ├── url.routes.js
│   │       ├── analytics.routes.js
│   │       └── auth.routes.js ← NEW
│   ├── controllers/
│   │   ├── url.controller.js (updated – now uses req.user)
│   │   ├── analytics.controller.js
│   │   └── auth.controller.js ← NEW
│   ├── middleware/
│   │   ├── rateLimiter.js
│   │   ├── urlChecker.js
│   │   └── auth.middleware.js ← NEW
│   └── services/
│       ├── groq.service.js
│       ├── qr.service.js ← NEW
│       └── geo.service.js
├── client/
│   ├── index.html
│   ├── login.html ← NEW
│   └── register.html ← NEW
├── .env
├── .env.example
├── package.json
└── README.md
```

2. INSTALLATION

```
mkdir smartlink-ai && cd smartlink-ai
npm init -y
npm install express mongoose ioredis nanoid@3 axios dotenv express-rate-limit rate-limit-redis groq-sdk bcryptjs jsonwebtoken
npm install --save-dev nodemon
```

Package purposes:

Package	Purpose
---------	---------

bcryptjs	Hash passwords before storing in DB
jsonwebtoken	Generate and verify JWT tokens
qrcode	Generate QR code as base64 image or PNG file
nanoid@3	Generate short unique IDs (use v3, not v4 — v4 is ESM only)

3. .env FILE

```
MONGO_URI=mongodb://localhost:27017/smartlink
REDIS_URL=redis://localhost:6379
GROQ_API_KEY=your_groq_key_here
SAFE_BROWSING_API_KEY=your_google_key_here
JWT_SECRET=your_super_secret_jwt_key_here_make_it_long
JWT_EXPIRES_IN=7d
BASE_URL=http://localhost:5000
PORT=5000
```

JWT_SECRET — make this a long random string (e.g. 64 random characters). Never commit the real value to GitHub.

4. MODELS

models/User.js — NEW

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true
  },
  password: {
    type: String,
    required: true,
    minlength: 6
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

// Hash password before saving
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// Compare password method
userSchema.methods.comparePassword = async function (candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};
```

```
};
```

```
module.exports = mongoose.model('User', userSchema);
```

models/Url.js — UPDATED (added userId field)

```
const mongoose = require('mongoose');
```

```
const clickSchema = new mongoose.Schema({
  timestamp: { type: Date, default: Date.now },
  ip: String,
  device: String,
  browser: String,
  country: String
});
```

```
const urlSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true // every link is owned by a user
  },
  originalUrl: { type: String, required: true },
  shortCode: { type: String, required: true, unique: true },
  customAlias: { type: String, default: null },
  title: { type: String, default: '' },
  description: { type: String, default: '' },
  qrCode: { type: String, default: '' }, // base64 QR image ← NEW
  createdAt: { type: Date, default: Date.now },
  expiresAt: { type: Date, default: null },
  clicks: [clickSchema]
});
```

```
urlSchema.index({ shortCode: 1 });
```

```
urlSchema.index({ userId: 1 }); // fast lookup of all user's links
```

```
module.exports = mongoose.model('Url', urlSchema);
```

5. SERVICES

services/qr.service.js — NEW

```
const QRCode = require('qrcode');
```

```
// Returns base64 string – embedded directly in JSON response
```

```
// Frontend displays it as: 
```

```
const generateQRCode = async (url) => {
  try {
    const qrBase64 = await QRCode.toDataURL(url, {
      errorCorrectionLevel: 'H', // high error correction
      type: 'image/png',
      width: 300,
      margin: 2,
      color: {
        dark: '#16213e', // dark navy dots
        light: '#ffffff' // white background
      }
    });
  } catch (err) {
    console.error('QR generation error:', err);
    return '';
  }
};
```

```
module.exports = { generateQRCode };
```

services/groq.service.js — unchanged from v1

```
const Groq = require('groq-sdk');
const groq = new Groq({ apiKey: process.env.GROQ_API_KEY });

const generateMetadata = async (url) => {
  try {
    const response = await groq.chat.completions.create({
      model: 'llama-3.1-8b-instant',
      messages: [{
        role: 'user',
        content: `Given this URL: ${url}, generate a short title (max 8 words) and
one sentence description. Respond in pure JSON only: {"title": "...", "description": "..."}`
      }]
    });
    return JSON.parse(response.choices[0].message.content);
  } catch (err) {
    return { title: 'Untitled Link', description: 'No description available' };
  }
};

const generateAnalyticsSummary = async (data) => {
  try {
    const response = await groq.chat.completions.create({
      model: 'llama-3.1-8b-instant',
      messages: [{
        role: 'user',
        content: `Given this analytics: ${JSON.stringify(data)}, write a 3 sentence
plain English summary covering total clicks, top device type, and peak activity day.`
      }]
    });
    return response.choices[0].message.content;
  } catch (err) {
    return 'Analytics summary unavailable';
  }
};

module.exports = { generateMetadata, generateAnalyticsSummary };
```

6. MIDDLEWARE

middleware/auth.middleware.js — NEW

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  try {
    // Get token from Authorization header: "Bearer <token>"
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ error: 'Not authorized. No token provided.' });
    }

    const token = authHeader.split(' ')[1];

    // Verify token
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // Attach user to request (excluding password)
    req.user = await User.findById(decoded.id).select('-password');
    if (!req.user) {

```

```

    return res.status(401).json({ error: 'User no longer exists.' });
  }

  next();
} catch (err) {
  return res.status(401).json({ error: 'Invalid or expired token.' });
}
};

module.exports = { protect };

```

middleware/rateLimiter.js — unchanged

```

const rateLimit = require('express-rate-limit');
const { RedisStore } = require('rate-limit-redis');
const redis = require('../config/redis');

const limiter = rateLimit({
  windowMs: 60 * 1000,
  max: 10,
  store: new RedisStore({ sendCommand: (...args) => redis.call(...args) }),
  message: { error: 'Too many requests. Try again in a minute.' }
});

module.exports = limiter;

```

middleware/urlChecker.js — unchanged

```

const axios = require('axios');

const checkMalicious = async (req, res, next) => {
  const { originalUrl } = req.body;
  try {
    const response = await axios.post(
      `https://safebrowsing.googleapis.com/v4/threatMatches:find?key=${process.env.SAFE_BROWSING_API_KEY}`,
      {
        client: { clientId: 'smartlink', clientVersion: '1.0' },
        threatInfo: {
          threatTypes: ['MALWARE', 'SOCIAL_ENGINEERING', 'UNWANTED_SOFTWARE'],
          platformTypes: ['ANY_PLATFORM'],
          threatEntryTypes: ['URL'],
          threatEntries: [{ url: originalUrl }]
        }
      }
    );
    if (response.data.matches) {
      return res.status(400).json({ error: 'Malicious URL detected.' });
    }
    next();
  } catch (err) { next(); }
};

module.exports = checkMalicious;

```

7. CONTROLLERS

controllers/auth.controller.js — NEW

```

const jwt = require('jsonwebtoken');
const User = require('../models/User');

// Helper: generate JWT token
const generateToken = (userId) => {
  return jwt.sign({ id: userId }, process.env.JWT_SECRET, {

```

```

    expiresIn: process.env.JWT_EXPIRES_IN || '7d'
  });
};

// POST /api/auth/register
const register = async (req, res) => {
  try {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({ error: 'All fields are required' });
    }
    if (password.length < 6) {
      return res.status(400).json({ error: 'Password must be at least 6 characters' });
    }

    // Check if email already registered
    const existing = await User.findOne({ email });
    if (existing) {
      return res.status(400).json({ error: 'Email already registered' });
    }

    // Create user – password is hashed in User model pre-save hook
    const user = await User.create({ name, email, password });
    const token = generateToken(user._id);

    res.status(201).json({
      message: 'Account created successfully',
      token,
      user: { id: user._id, name: user.name, email: user.email }
    });
  } catch (err) {
    res.status(500).json({ error: 'Server error' });
  }
};

// POST /api/auth/login
const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password) {
      return res.status(400).json({ error: 'Email and password are required' });
    }

    // Find user
    const user = await User.findOne({ email });
    if (!user) {
      return res.status(401).json({ error: 'Invalid email or password' });
    }

    // Compare password
    const isMatch = await user.comparePassword(password);
    if (!isMatch) {
      return res.status(401).json({ error: 'Invalid email or password' });
    }

    const token = generateToken(user._id);

    res.json({
      message: 'Login successful',
      token,
      user: { id: user._id, name: user.name, email: user.email }
    });
  } catch (err) {

```

```

    res.status(500).json({ error: 'Server error' });
  }
};

// GET /api/auth/me – get logged in user profile
const getMe = async (req, res) => {
  res.json({
    user: { id: req.user._id, name: req.user.name, email: req.user.email }
  });
};

module.exports = { register, login, getMe };

```

controllers/url.controller.js — UPDATED (auth + QR code)

```

const { nanoid } = require('nanoid');
const Url = require('../models/Url');
const redis = require('../config/redis');
const { generateMetadata } = require('../services/groq.service');
const { generateQRCode } = require('../services/qr.service');

// POST /api/shorten (protected route – requires login)
const shortenUrl = async (req, res) => {
  try {
    const { originalUrl, customAlias, expiresIn } = req.body;

    if (!originalUrl) {
      return res.status(400).json({ error: 'URL is required' });
    }

    const shortCode = customAlias || nanoid(6);

    // Check alias availability
    const existing = await Url.findOne({ shortCode });
    if (existing) {
      return res.status(400).json({ error: 'Alias already taken' });
    }

    const shortUrl = `${process.env.BASE_URL}/${shortCode}`;

    // Run Groq metadata + QR code generation in parallel (faster)
    const [{ title, description }, qrCode] = await Promise.all([
      generateMetadata(originalUrl),
      generateQRCode(shortUrl) // QR points to the short URL
    ]);

    const expiresAt = expiresIn
      ? new Date(Date.now() + expiresIn * 24 * 60 * 60 * 1000)
      : null;

    const url = await Url.create({
      userId: req.user._id, // link owned by logged in user
      originalUrl,
      shortCode,
      customAlias: customAlias || null,
      title,
      description,
      qrCode,
      expiresAt
    });

    // Cache in Redis – 24 hours TTL
    await redis.setex(shortCode, 86400, originalUrl);

    res.status(201).json({ shortUrl, shortCode, title, description, qrCode });
  }
};

```

```

    } catch (err) {
      console.error(err);
      res.status(500).json({ error: 'Server error' });
    }
  };

// GET /:shortCode - public redirect (no auth needed)
const redirectUrl = async (req, res) => {
  try {
    const { shortCode } = req.params;

    // Redis cache first
    const cached = await redis.get(shortCode);
    if (cached) {
      logClick(shortCode, req);
      return res.redirect(cached);
    }

    // MongoDB fallback
    const url = await Url.findOne({ shortCode });
    if (!url) return res.status(404).json({ error: 'Link not found' });

    if (url.expiresAt && url.expiresAt < new Date()) {
      return res.status(410).json({ error: 'This link has expired' });
    }

    await redis.setex(shortCode, 86400, url.originalUrl);
    logClick(shortCode, req);
    res.redirect(url.originalUrl);
  } catch (err) {
    res.status(500).json({ error: 'Server error' });
  }
};

// GET /api/my-links - get all links for logged in user
const getMyLinks = async (req, res) => {
  try {
    const urls = await Url.find({ userId: req.user._id })
      .select('-clicks') // exclude click array for performance
      .sort({ createdAt: -1 });
    res.json({ count: urls.length, urls });
  } catch (err) {
    res.status(500).json({ error: 'Server error' });
  }
};

// DELETE /api/url/:shortCode - delete own link
const deleteUrl = async (req, res) => {
  try {
    const { shortCode } = req.params;
    const url = await Url.findOne({ shortCode });

    if (!url) return res.status(404).json({ error: 'Link not found' });

    // Only owner can delete
    if (url.userId.toString() !== req.user._id.toString()) {
      return res.status(403).json({ error: 'Not authorized to delete this link' });
    }

    await Url.deleteOne({ shortCode });
    await redis.del(shortCode);

    res.json({ message: 'Link deleted successfully' });
  } catch (err) {

```



```

    res.status(500).json({ error: 'Server error' });
  }
};

// Internal – log click async
const logClick = async (shortCode, req) => {
  try {
    const ua = req.headers['user-agent'] || '';
    const device = ua.includes('Mobile') ? 'mobile' : 'desktop';
    const browser = ua.includes('Chrome') ? 'Chrome'
      : ua.includes('Firefox') ? 'Firefox'
      : ua.includes('Safari') ? 'Safari' : 'Other';

    await Url.updateOne({ shortCode }, {
      $push: { clicks: { timestamp: new Date(), ip: req.ip, device, browser } }
    });
  } catch (err) {
    console.error('Click log error:', err);
  }
};

module.exports = { shortenUrl, redirectUrl, getMyLinks, deleteUrl };

```

controllers/analytics.controller.js — unchanged

```

const Url = require('../models/Url');
const { generateAnalyticsSummary } = require('../services/groq.service');

const getAnalytics = async (req, res) => {
  try {
    const { shortCode } = req.params;
    const url = await Url.findOne({ shortCode });
    if (!url) return res.status(404).json({ error: 'Link not found' });

    // Only owner can view analytics
    if (url.userId.toString() !== req.user._id.toString()) {
      return res.status(403).json({ error: 'Not authorized' });
    }

    const totalClicks = url.clicks.length;
    const devices = url.clicks.reduce((acc, c) => {
      acc[c.device] = (acc[c.device] || 0) + 1; return acc; }, {});
    const browsers = url.clicks.reduce((acc, c) => {
      acc[c.browser] = (acc[c.browser] || 0) + 1; return acc; }, {});
    const byDate = url.clicks.reduce((acc, c) => {
      const date = new Date(c.timestamp).toISOString().split('T')[0];
      acc[date] = (acc[date] || 0) + 1; return acc; }, {});

    const analyticsData = { title: url.title, totalClicks, devices, browsers, byDate };
    const aiSummary = await generateAnalyticsSummary(analyticsData);

    res.json({ ...analyticsData, aiSummary });
  } catch (err) {
    res.status(500).json({ error: 'Server error' });
  }
};

module.exports = { getAnalytics };

```

8. ROUTES

routes/auth.routes.js — NEW

```

const express = require('express');

```

```

const router = express.Router();
const { register, login, getMe } = require('../controllers/auth.controller');
const { protect } = require('../middleware/auth.middleware');

router.post('/register', register);
router.post('/login', login);
router.get('/me', protect, getMe); // protected

module.exports = router;

```

routes/url.routes.js — UPDATED

```

const express = require('express');
const router = express.Router();
const { shortenUrl, getMyLinks, deleteUrl } = require('../controllers/url.controller');
const checkMalicious = require('../middleware/urlChecker');
const limiter = require('../middleware/rateLimiter');
const { protect } = require('../middleware/auth.middleware');

// All URL routes require login
router.post('/shorten', protect, limiter, checkMalicious, shortenUrl);
router.get('/my-links', protect, getMyLinks);
router.delete('/url/:shortCode', protect, deleteUrl);

module.exports = router;

```

routes/analytics.routes.js — UPDATED (now protected)

```

const express = require('express');
const router = express.Router();
const { getAnalytics } = require('../controllers/analytics.controller');
const { protect } = require('../middleware/auth.middleware');

router.get('/:shortCode', protect, getAnalytics);

module.exports = router;

```

9. server/index.js — UPDATED

```

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');
const { redirectUrl } = require('../controllers/url.controller');

const app = express();

app.use(cors());
app.use(express.json({ limit: '10kb' }));
app.use(express.static('client'));

connectDB();

// Public redirect route — MUST be before other routes
app.get('/:shortCode', redirectUrl);

// API routes
app.use('/api/auth', require('../routes/auth.routes'));
app.use('/api', require('../routes/url.routes'));
app.use('/api/analytics', require('../routes/analytics.routes'));

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));

```

Add cors to dependencies: `npm install cors`

10. COMPLETE API ENDPOINTS

Method	Route	Auth?	Purpose
POST	/api/auth/register	No	Create new account
POST	/api/auth/login	No	Login, get JWT token
GET	/api/auth/me	Yes	Get logged in user profile
POST	/api/shorten	Yes	Create short URL + QR code
GET	/api/my-links	Yes	Get all my links
DELETE	/api/url/:shortCode	Yes	Delete my link
GET	/api/analytics/:shortCode	Yes	Get analytics + AI summary
GET	/:shortCode	No	Redirect to original URL

11. HOW AUTH FLOW WORKS

REGISTER:

User sends name + email + password
→ Backend hashes password with bcrypt (12 salt rounds)
→ Saves User to MongoDB
→ Generates JWT token (expires in 7 days)
→ Returns token + user info

LOGIN:

User sends email + password
→ Backend finds user by email
→ `bcrypt.compare(inputPassword, hashedPassword)`
→ If match → generate JWT token
→ Return token + user info

PROTECTED ROUTE:

Frontend sends: `Authorization: Bearer <token>`
→ `auth.middleware.js` extracts token
→ `jwt.verify()` checks signature + expiry
→ Finds user from decoded ID
→ Attaches to `req.user`
→ Controller uses `req.user._id` for ownership checks

12. HOW QR CODE WORKS

GENERATION:

When user shortens a URL:
→ `generateQRcode(shortUrl)` called in parallel with Groq
→ `qrcode` library generates PNG as base64 data URL
→ Stored in `Url.qrCode` field in MongoDB
→ Returned in API response as base64 string

DISPLAY ON FRONTEND:

```

```

→ Browser renders QR image directly from base64
→ User can right-click and save as image
→ Or use download button (shown in frontend guide)

WHY BASE64 VS FILE:

→ Base64 stored in DB – no need for file storage
→ Works without S3, Cloudinary, or disk storage

- Slightly larger storage but simpler architecture
- Fine for a resume project at this scale

13. COMPLETE package.json

```
{
  "name": "smartlink-ai",
  "version": "2.0.0",
  "description": "AI-powered URL shortener with auth, QR codes, and analytics",
  "main": "server/index.js",
  "scripts": {
    "start": "node server/index.js",
    "dev": "nodemon server/index.js"
  },
  "dependencies": {
    "axios": "^1.6.0",
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "express-rate-limit": "^7.1.5",
    "groq-sdk": "^0.3.3",
    "ioredis": "^5.3.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.0.3",
    "nanoid": "^3.3.7",
    "qrcode": "^1.5.3",
    "rate-limit-redis": "^4.2.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.2"
  }
}
```

14. INTERVIEW TALKING POINTS

- "I used bcrypt with 12 salt rounds — industry standard for password hashing"
- "JWT stored on frontend localStorage — stateless auth, no session on server"
- "QR code generated in parallel with AI metadata using Promise.all — saves ~500ms"
- "Each URL has a userId field — ownership enforced at controller level before delete/analytics"
- "Redis still serves all redirects — auth only applies to creation/management routes"
- "QR stored as base64 in MongoDB — avoids need for file storage service at this scale"